

Deep Learning for NLP

Student name: *Georgios Xydias*
ID: 7115152400007

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*
Semester: *Spring Semester 2025-2026*

Response Clarity Classification with Transformers

Contents

1	Abstract	3
2	Data processing and analysis	3
2.1	Dataset Description	3
2.2	Exploratory Data Analysis	3
2.3	Text Preprocessing	4
2.4	Input Construction and Tokenization	5
2.5	Data Partitioning	5
3	Model Descriptions	6
3.1	BERT-base-uncased	6
3.2	DistilBERT-base-uncased	6
3.3	DeBERTa-v3-base	6
4	Experiments	7
4.1	Baseline Models (v0)	7
4.2	Addressing Class Imbalance (v1)	7
4.3	Hyperparameter Tuning (v2)	8
4.4	Warmup and Input Formulation	8
4.5	Results Summary	8
5	Evaluation	10
5.1	Learning Curves	10
5.2	Confusion Matrices	11
5.3	Per-Class Performance	12
5.4	Test Set Performance	12
6	Error Analysis	13
6.1	Hardest Class to Predict	13
6.2	Performance Across Data Subgroups	13
6.3	Concrete Error Examples	14
6.4	Model-Specific Failure Modes	15

6.5	Fundamental Sources of Error and Ideal Model	15
7	Discussion	16
7.1	Results Analysis	16
7.2	Comparison with Classical ML Approaches	17
7.3	Submitted Configuration	17
7.4	Future Work	19
A	Additional Experiments	21
B	Hyperparameter Search Space	22

1. Abstract

This report presents our approach to response clarity classification in political question-answer pairs, as part of SemEval 2026 Task 6 [12]. The dataset, introduced by [11], consists of political interviews with U.S. presidents, where each answer is labeled as *Clear Reply*, *Ambivalent*, or *Clear Non-Reply*. We fine-tune three pretrained transformer models using PyTorch [8] and the Hugging Face Transformers library [14]: BERT-base-uncased [1], DistilBERT-base-uncased [9], and DeBERTa-v3-base [3, 2]. All models are trained with custom explicit training loops and optimized using AdamW [6]. We conduct a systematic experimental pipeline that includes class imbalance handling, hyperparameter tuning, learning rate warmup, and input formulation experiments. We compare the three models across all experiments and analyze their performance differences, failure modes, and the effect of each optimization step. Our approach follows the methodology outlined in [5] and the relevant chapters of [4]. All implementation details, additional figures, and intermediate results for every experiment presented in this report are available in the accompanying Jupyter notebooks.

2. Data processing and analysis

2.1. Dataset Description

We use the dataset introduced by [11] for SemEval 2026 Task 6 [12], which consists of political interviews with U.S. presidents. Each record contains the original journalist turn (`interview_question`), the politician’s response (`interview_answer`), annotation information, metadata (e.g., title, date, president, URL), and auxiliary fields such as GPT-3.5 generated summaries. Because a single journalist turn may contain multiple questions, the dataset provides a segmented question field containing exactly one question. The core supervised instances in this work are therefore (`question`, `interview_answer`) pairs. The prediction target is `clarity_label`, a three-class label: *Clear Reply*, *Ambivalent*, and *Clear Non-Reply*. The dataset is available on Hugging Face under the identifier `ailsntua/QEvasion`, and provides two splits: 3,448 training samples and 308 test samples.

2.2. Exploratory Data Analysis

Before building any model, we performed an exploratory data analysis (EDA) to better understand the dataset. We examined the class distribution and text length statistics, as both directly affect modeling decisions.

The class distribution is highly imbalanced: *Ambivalent* accounts for 59% of the training samples, *Clear Reply* for 31%, and *Clear Non-Reply* for only 10% (Figure 1). This imbalance is important, as a basic model would simply predict the majority class and achieve high accuracy while failing to detect minority classes. The class distribution is also consistent between the training and test splits.

Text length analysis (Figure 3) shows that questions are short and consistent (mean 14.5 words, median 12), while answers are heavily right-skewed (mean 293.6 words, median 207, max 2,117). The combined input length has a mean of 308 words with a long tail extending beyond 2,000 words. Since transformer models such as BERT have a maximum sequence length of 512 tokens, long answers will be truncated during

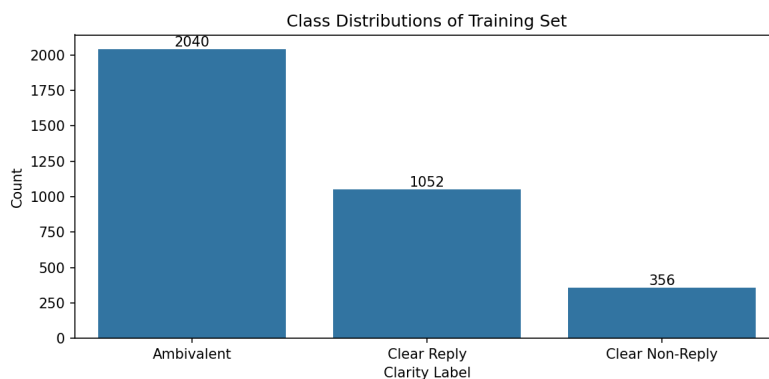


Figure 1: Class distribution in the training set.

tokenization, potentially losing useful information for classification. Additionally, text length varies across classes (Figure 2): *Ambivalent* answers tend to be the longest (mean 331 words), followed by *Clear Reply* (272 words), and *Clear Non-Reply* (137 words), suggesting that politicians may use verbosity as a strategy when providing only a partial or indirect response.

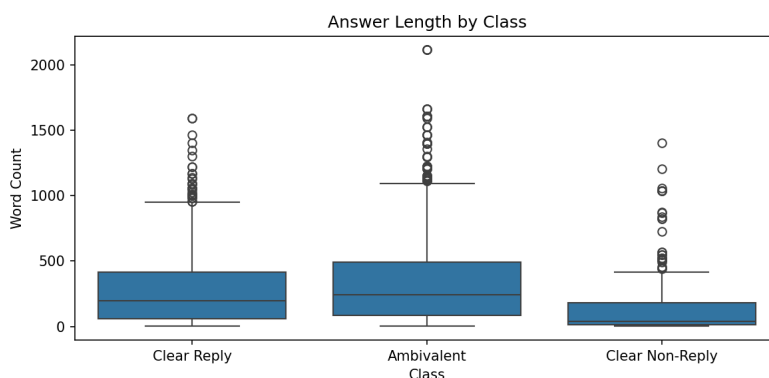


Figure 2: Answer length distribution per class.

The combined length distribution (Figure 3) is particularly relevant for transformer-based models, as it directly informs the choice of maximum sequence length during tokenization. Only 35.9% of examples fit within 128 words, 54.3% within 256 words, and 78.9% within 512 words. This means that regardless of the chosen `max_length` setting, a significant portion of answers will be truncated.

2.3. Text Preprocessing

Minimal text preprocessing was applied, as pretrained transformer models operate on raw text and rely on their own pretrained tokenizers for subword tokenization. We applied two preprocessing steps:

Deduplication. We identified and removed 58 duplicate rows (1.68%) from the training set, reducing it from 3,448 to 3,390 samples. This prevents the same example from appearing in both training and validation sets, which would lead to unreliable performance estimates.

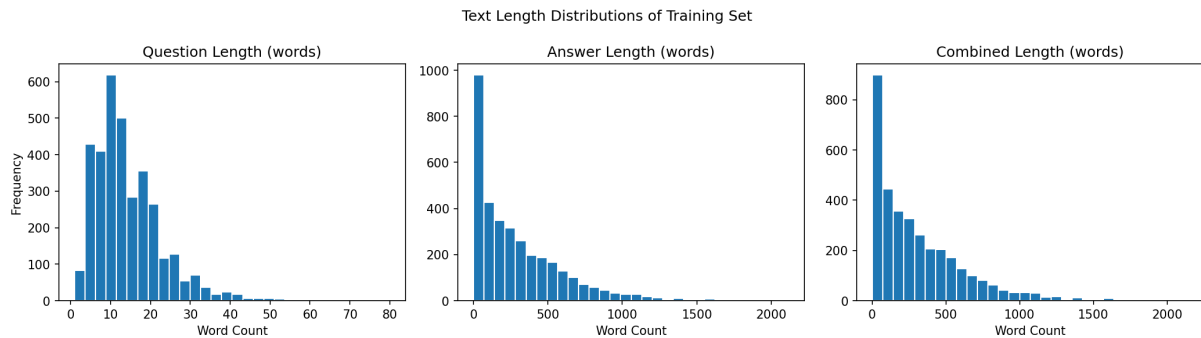


Figure 3: Text length distributions of questions, answers, and combined input in the training set.

Label Encoding. The categorical `clarity_label` field was mapped to integer values: *Clear Reply* $\rightarrow$ 0, *Ambivalent* $\rightarrow$ 1, and *Clear Non-Reply* $\rightarrow$ 2.

2.4. Input Construction and Tokenization

Each input instance consists of a question-answer pair. We use each model’s corresponding pretrained tokenizer to jointly encode both sequences. The tokenizer is called with the question as the first argument and the answer as the second, which automatically constructs the input in the following format and handles all special token insertion internally:

[CLS] question [SEP] answer [SEP]

We set `max_length=512` for all three models, which is the maximum sequence length supported by BERT and DistilBERT. DeBERTa supports significantly longer sequences, however, we use 512 as the default for a fair comparison and separately test `max_length=768` for DeBERTa in Section 4.4. All inputs are padded to this fixed length (`padding="max_length"`) and longer sequences are truncated (`truncation=True`). The tokenizer returns three tensors per input: `input_ids`, `attention_mask`, and where applicable, `token_type_ids`. Model-specific differences in how these inputs are handled are described in Section 3.

2.5. Data Partitioning

We split the 3,390 deduplicated training samples into a training set (85%) and a validation set (15%) using a random split, resulting in 2,881 training and 509 validation samples. This ratio provides a sufficiently large training set for fine-tuning while keeping enough validation samples for reliable evaluation. The split indices were saved to disk and reused across all three model notebooks to guarantee a fair comparison. The test set (308 samples) was kept completely untouched until final prediction. A fixed random seed (`SEED=42`) was used throughout all experiments to ensure reproducibility. We also experimented with stratified splitting to preserve class proportions across splits, but this did not improve validation performance.

3. Model Descriptions

For BERT, the model and tokenizer were loaded using `BertForSequenceClassification` and `BertTokenizer` respectively, following the approach demonstrated in the course lab notebook. For DistilBERT and DeBERTa, we used `AutoModelForSequenceClassification` and `AutoTokenizer` from the Hugging Face Transformers library [14]. In all cases, models were initialized with `num_labels=3`, which adds a linear classification head on top of each pretrained model's output representation. All models are based on the Transformer encoder architecture [13].

Table 1 summarizes the key architectural differences between the three models.

	BERT	DistilBERT	DeBERTa-v3
Transformer layers	12	6	12
Attention mechanism	Standard	Standard	Disentangled
Hidden size	768	768	768
Parameters	109.5M	67M	184M
Tokenizer	WordPiece	WordPiece	SentencePiece
Vocabulary size	30,522	30,522	128,000
Token type IDs	Yes	No	No
Pooler layer	Yes	No	Yes
Max sequence length	512	512	>512

Table 1: Architectural comparison of the three required models.

3.1. BERT-base-uncased

BERT (Bidirectional Encoder Representations from Transformers) [1] consists of 12 transformer layers (`BertLayer`) with a hidden size of 768 and approximately 109.5M parameters. It uses a WordPiece tokenizer with a vocabulary of 30,522 tokens. BERT encodes the input pair as [CLS] question [SEP] answer [SEP] and uses `token_type_ids` to distinguish between the two segments. The [CLS] token representation is passed through a pooler layer (a linear layer followed by tanh activation) before being fed to the classification head.

3.2. DistilBERT-base-uncased

DistilBERT [9] is a compressed version of BERT, obtained through knowledge distillation during pretraining. It retains BERT's hidden size of 768 but uses only 6 transformer layers (`TransformerBlock`), reducing the parameter count to approximately 67M. This makes it roughly 40% faster than BERT during training and inference. DistilBERT uses the same WordPiece tokenizer as BERT but does not support `token_type_ids` and does not include a pooler layer. In our implementation, `token_type_ids` were excluded from the model input for DistilBERT.

3.3. DeBERTa-v3-base

DeBERTa (Decoding-enhanced BERT with Disentangled Attention) [3] introduces

two key innovations: a disentangled attention mechanism that separately encodes content and position information, and an enhanced mask decoder. The v3 variant [2] further improves pretraining efficiency through a more advanced training strategy. DeBERTa-v3-base has 12 transformer layers (DebertaV2Layer) with a hidden size of 768 and approximately 184M parameters. Despite having the same depth as BERT, the higher parameter count is due to its larger vocabulary: it uses a SentencePiece tokenizer (Unigram algorithm) with a vocabulary of 128,000 tokens, compared to BERT's 30,522. DeBERTa natively supports sequence lengths well beyond 512 tokens.

4. Experiments

We conducted a systematic series of experiments for each of the three models, progressively addressing observed weaknesses. All experiments share the same AdamW optimizer [6] and use Macro F1 on the validation set as the primary evaluation metric. Each experiment builds on the findings of the previous one. All models were developed and trained on Google Colab using an NVIDIA A100-SXM4-40GB GPU. For the Kaggle submission, notebooks were executed on Kaggle's NVIDIA T4 $\times 2$ GPUs, which may cause minor variations in reported metrics due to differences in hardware and floating-point precision.

4.1. Baseline Models (v0)

We trained all three models with default settings to establish a performance reference point: learning rate 2×10^{-5} , batch size of 16, no weight decay, no class weighting, no learning rate warmup, and 4 training epochs with the input formatted as a question-answer text pair.

The baseline results revealed two key patterns. First, all three models show signs of overfitting: training loss decreases steadily across epochs while validation loss begins to increase after epoch 2–3, particularly for BERT (generalization gap of 0.41 at epoch 4). DistilBERT overfits less aggressively due to its reduced capacity, while DeBERTa maintains the smallest generalization gap throughout, suggesting better built-in regularization. Second, all baselines achieve similar Macro F1 scores (BERT: 0.617, DistilBERT: 0.607, DeBERTa: 0.643), with DeBERTa showing the strongest baseline performance. These observations motivate the use of class weighting, weight decay, early stopping, and learning rate tuning in subsequent experiments.

4.2. Addressing Class Imbalance (v1)

The baseline models used standard cross-entropy loss, which treats all classes equally. Given the severe class imbalance (59% Ambivalent, 31% Clear Reply, 10% Clear Non-Reply), this setup biases the models toward predicting the majority class. To address this, we introduced weighted cross-entropy loss using scikit-learn's `compute_class_weight` with the 'balanced' strategy [10], which assigns higher weights to rarer classes so the model pays more attention to them. The computed weights were passed to PyTorch's `nn.CrossEntropyLoss` as the `weight` parameter.

Class weighting improved Macro F1 for BERT (0.617 $\rightarrow$ 0.623) and DeBERTa (0.643 $\rightarrow$ 0.658), while DistilBERT remained essentially unchanged (0.607). The improvement

was modest for BERT and DeBERTa, and null for DistilBERT, suggesting that class imbalance is only part of the problem and further tuning is needed.

4.3. Hyperparameter Tuning (v2)

Building on the class-weighted setup, we performed a grid search over learning rates and weight decay values. For BERT and DistilBERT, we searched over learning rates $\{5 \times 10^{-5}, 3 \times 10^{-5}, 2 \times 10^{-5}\}$ and weight decay $\{0.0, 0.01\}$ with a fixed batch size of 16. For DeBERTa, we used a lower learning rate range $\{3 \times 10^{-5}, 2 \times 10^{-5}, 1 \times 10^{-5}\}$ and additionally explored batch sizes of 8 and 16, as its larger model size required more careful tuning. Batch size 32 was also tested but caused out-of-memory errors on the available GPU for both BERT and DeBERTa. All configurations used class weighting enabled.

Hyperparameter tuning yielded the largest improvements across all models. The best configuration for both BERT and DistilBERT was $\text{lr}=5 \times 10^{-5}$ with weight decay 0.01, improving Macro F1 to 0.660 and 0.620 respectively. DeBERTa achieved its best result of 0.670 with $\text{lr}=2 \times 10^{-5}$ and no weight decay, suggesting that DeBERTa’s architecture is already well-regularized and does not benefit from additional weight decay.

4.4. Warmup and Input Formulation

We also tried two additional approaches to further improve performance. First, we experimented with learning rate warmup, where the learning rate gradually increases from zero during the initial training steps before following the linear decay schedule. The motivation was to allow the pretrained weights to adjust more gently at the beginning of training, potentially reducing early instability.

Second, we tested an alternative input formulation. Instead of passing the question and answer as a sentence pair (which lets the tokenizer handle segment separation automatically), we concatenated them into a single string with an explicit separator token, following the input construction approach from our earlier work on this dataset:

question [SEP] interview_answer

The goal was to examine whether the model’s handling of the two-segment structure affects classification performance.

Additionally, for DeBERTa we tested $\text{max_length}=768$ to take advantage of its support for longer sequences, but this decreased Macro F1 from 0.67 to 0.65, suggesting that the additional tokens introduce noise rather than useful signal.

Neither technique produced consistent improvements across all three models. Warmup slightly helped some configurations but did not outperform the hyperparameter-tuned setup, while the alternative input formulation had a mixed effect depending on the model. For each model’s final configuration, we kept only the settings that showed a positive effect and discarded the rest.

4.5. Results Summary

Based on all previous experiments, we assembled the best configuration for each model by keeping only the settings that showed a positive effect: the optimal learning rate, weight decay, batch size, and class weighting from hyperparameter tuning

(v2), plus warmup and input formulation only where they helped. Each model was then retrained from scratch using its final configuration. Table 2 presents the selected configurations and Table 3 summarizes the validation results across all experiments.

Model	LR	WD	BS	Weighted Loss	Warmup	Input	Epochs
BERT	5e-5	0.01	16	Yes	No	text_pair	2
DistilBERT	5e-5	0.01	16	Yes	No	text_pair	3
DeBERTa	2e-5	0.0	16	Yes	10%	concat	4

Table 2: Final configuration per model used for retrain.

Model	Experiment	Macro F1	Weighted F1
BERT	v0: Baseline	0.617	0.663
	v1: +Class weighting	0.622	0.627
	v2: +HP tuning	0.660	0.680
	Final retrain	0.677	0.677
DistilBERT	v0: Baseline	0.607	0.651
	v1: +Class weighting	0.607	0.610
	v2: +HP tuning	0.620	0.650
	Final retrain	0.626	0.649
DeBERTa	v0: Baseline	0.643	0.670
	v1: +Class weighting	0.658	0.671
	v2: +HP tuning	0.670	0.690
	Final retrain	0.669	0.665

Table 3: Validation results across all experiments for each model.

Across all three models, hyperparameter tuning (v2) produced the largest and most consistent improvement. Class weighting helped BERT and DeBERTa but had little effect on DistilBERT. DeBERTa was the only model that benefited from warmup and the concatenated input formulation, while BERT and DistilBERT performed best with the standard text pair format and no warmup. For DeBERTa, warmup (10%) and the concat formulation each showed a small positive effect and were retained in the final configuration. The best overall validation Macro F1 was achieved by BERT (0.677), closely followed by DeBERTa (0.669), with DistilBERT trailing at 0.626 as expected given its reduced capacity. However, BERT’s higher Macro F1 comes with a larger generalization gap compared to DeBERTa, which maintained more stable training throughout all experiments. The test set evaluation in Section 5.4 provides a more definitive comparison.

We also experimented with larger model variants to investigate the effect of model capacity: BERT-large (335M params), RoBERTa-base (125M params), RoBERTa-large (355M params), and DeBERTa-v3-large (435M params). The best result among all models was achieved by DeBERTa-v3-large with a validation Macro F1 of 0.69, confirming that larger models can further improve performance on this task. However, these models required significantly longer training times (up to 1,365 seconds) and lower batch sizes due to memory constraints. A detailed comparison is provided in Appendix A.

In terms of training efficiency, DistilBERT was the fastest model at approximately 28 seconds per training epoch, followed by BERT at 58 seconds and DeBERTa at 1 minute 26 seconds, consistent with their respective model sizes.

5. Evaluation

5.1. Learning Curves

To assess training behavior and detect overfitting, we plotted the training and validation loss across epochs for both the baseline (v0) and final configurations of each model (Figures 4–6).

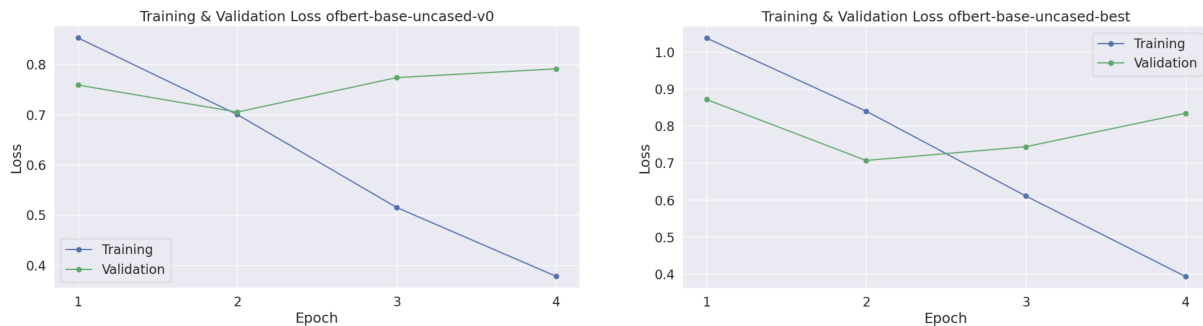


Figure 4: BERT learning curves: baseline (left) vs final configuration (right).

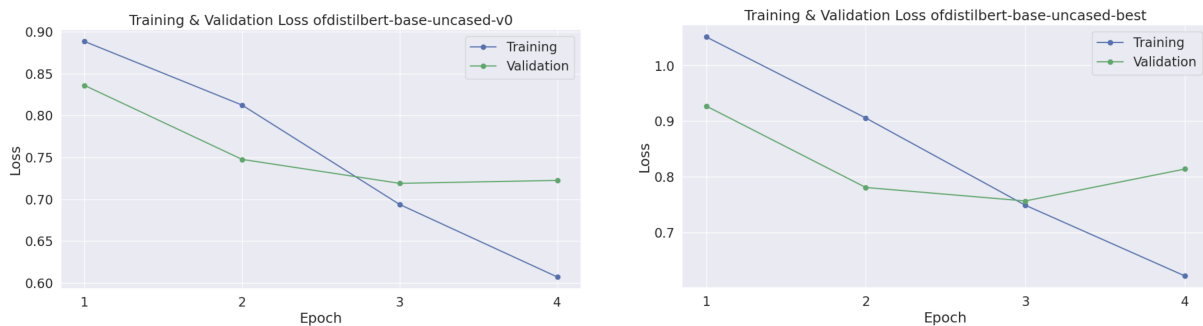


Figure 5: DistilBERT learning curves: baseline (left) vs final configuration (right).

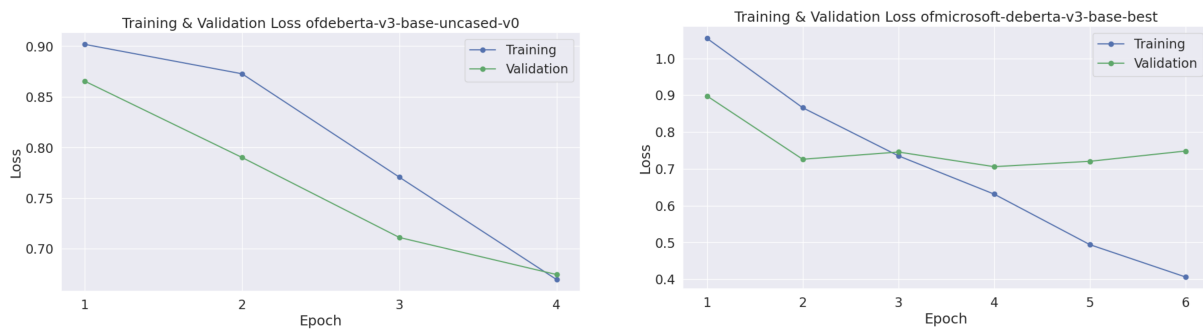


Figure 6: DeBERTa learning curves: baseline (left) vs final configuration (right).

In the baseline configurations (left), BERT and DistilBERT show a growing gap between training and validation loss, indicating overfitting. BERT exhibits the most aggressive overfitting, with validation loss increasing after epoch 2 while training loss continues to drop. DeBERTa shows the most stable baseline behavior, with both losses decreasing consistently through all 4 epochs. In the final configurations (right), the training loss starts higher due to class weighting and weight decay acting as regularizers. BERT and DistilBERT still overfit in later epochs, which is why early stopping selects epoch 2 and 3 respectively as the best checkpoints. DeBERTa’s final configuration runs for 6 epochs and shows the gap growing only in the last two epochs, with the best checkpoint at epoch 4. Overall, the combination of hyperparameter tuning and early stopping effectively controls overfitting across all models.

5.2. Confusion Matrices

Figures 7–9 show the confusion matrices for the final best configuration of each model on the validation set.

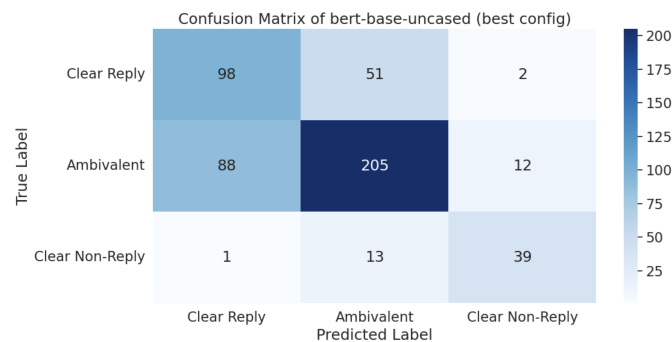


Figure 7: Confusion matrix for BERT (final configuration).

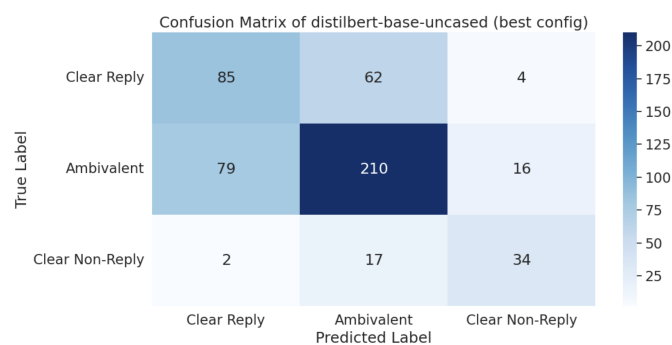


Figure 8: Confusion matrix for DistilBERT (final configuration).

All three models follow a similar pattern. *Ambivalent* is predicted most reliably due to its majority status, while *Clear Reply* and *Ambivalent* are frequently confused with each other. This is the dominant error across all models (BERT: 88+51, DistilBERT: 79+62, DeBERTa: 103+39 misclassifications in both directions). *Clear Non-Reply* achieves the highest precision across all models, with very few samples incorrectly predicted as Clear Non-Reply, but its recall varies: DeBERTa correctly identifies 40 out of 53 (75%), compared to BERT’s 39 (74%) and DistilBERT’s 34 (64%). A detailed analysis of these error patterns is provided in Section 6.

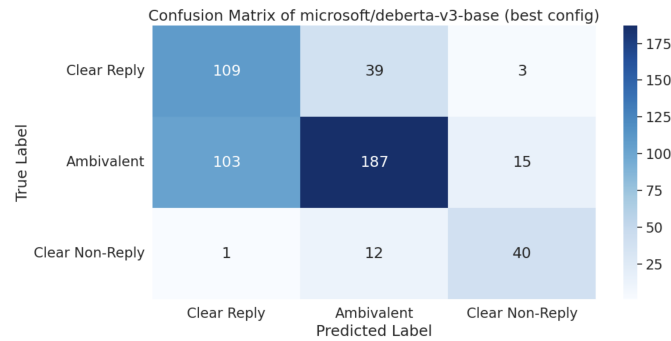


Figure 9: Confusion matrix for DeBERTa (final configuration).

5.3. Per-Class Performance

Table 4 reports the precision, recall, and F1 score per class for each model’s final configuration on the validation set.

Model	Class	Precision	Recall	F1
BERT	Clear Reply	0.52	0.65	0.58
	Ambivalent	0.76	0.67	0.71
	Clear Non-Reply	0.74	0.74	0.74
DistilBERT	Clear Reply	0.51	0.56	0.54
	Ambivalent	0.73	0.69	0.71
	Clear Non-Reply	0.63	0.64	0.64
DeBERTa	Clear Reply	0.51	0.72	0.60
	Ambivalent	0.79	0.61	0.69
	Clear Non-Reply	0.69	0.75	0.72

Table 4: Per-class precision, recall, and F1 for each model’s final configuration on the validation set.

BERT and DistilBERT achieve their highest F1 on *Ambivalent* (0.71), which benefits from being the majority class. DeBERTa, however, achieves its highest F1 on *Clear Non-Reply* (0.72), suggesting that its disentangled attention mechanism better captures the patterns of indirect responses. *Clear Reply* is consistently the hardest class to predict across all models (F1: 0.54–0.60), primarily due to low precision (0.51), indicating that many Ambivalent samples are incorrectly predicted as Clear Reply. DeBERTa achieves the best Clear Reply recall (0.72), while BERT leads on Clear Non-Reply F1 (0.74). This pattern is further analyzed in Section 6.

5.4. Test Set Performance

We evaluated each model’s final configuration on the test set (308 samples) using the same preprocessing and tokenization pipeline as during training. Table 5 reports the results.

All three models show a drop in Macro F1 from validation to test: BERT (0.68 → 0.61), DistilBERT (0.63 → 0.55), and DeBERTa (0.67 → 0.57). In contrast, Weighted F1 remains relatively stable or even improves: BERT (0.68 → 0.69), DistilBERT (0.65 →

Model	Macro F1	Weighted F1
BERT	0.61	0.69
DistilBERT	0.55	0.65
DeBERTa	0.57	0.67

Table 5: Test set performance for each model’s final configuration.

0.65), and DeBERTa (0.67 $\rightarrow$ 0.67). This divergence indicates that the models maintain strong performance on the majority class (*Ambivalent*) in the test set, but their ability to correctly predict minority classes (*Clear Reply* and *Clear Non-Reply*) degrades on unseen data. The gap likely caused by the small number of minority-class samples in the validation set (from our random, non-stratified split), which makes validation Macro F1 a noisy signal. Hyperparameter selection optimized for those noisy minority patterns, which did not generalize to the test set. This is analyzed further in Section 7.1.

6. Error Analysis

6.1. Hardest Class to Predict

As shown in Section 5.3, *Clear Reply* is consistently the hardest class to predict across all three models, with the lowest per-class F1 (0.54–0.60). Table 6 shows the most common misclassification pairs for each model.

True Label	Predicted	BERT	DistilBERT	DeBERTa
Ambivalent	Clear Reply	88	79	90
Clear Reply	Ambivalent	51	62	56
Clear Non-Reply	Ambivalent	13	17	17
Ambivalent	Clear Non-Reply	12	16	18
Clear Reply	Clear Non-Reply	2	4	4
Clear Non-Reply	Clear Reply	1	2	1

Table 6: Most common misclassification pairs across all models (validation set).

The dominant error across all models is the confusion between *Ambivalent* and *Clear Reply*, accounting for over 80% of all misclassifications. This is expected: both classes involve the politician providing meaningful content, making it difficult for the model to distinguish whether the response fully addresses the question or only partially does so. In contrast, *Clear Non-Reply* is rarely confused with *Clear Reply*, suggesting that the boundary between direct responses and complete evasions is relatively well-defined, while the boundary between partial and full responses is naturally unclear.

6.2. Performance Across Data Subgroups

We analyzed error rates across answer length and question length groups to identify whether certain input characteristics make classification harder. Table 7 and Table 8 show the results.

For answer length, all models perform best on short answers (<50 words) and worst on long answers (150–300 words). Interestingly, very long answers (>300 words)

Answer Length	Samples	BERT Acc.	DistilBERT Acc.	DeBERTa Acc.
Short (<50)	116	0.72	0.68	0.64
Medium (50–150)	106	0.65	0.59	0.61
Long (150–300)	111	0.61	0.58	0.58
Very Long (>300)	176	0.69	0.70	0.68

Table 7: Accuracy by answer length group across all models.

Question Length	Samples	BERT Acc.	DistilBERT Acc.	DeBERTa Acc.
Short (<10)	196	0.71	0.68	0.67
Medium (10–20)	216	0.65	0.59	0.59
Long (20–30)	70	0.64	0.69	0.61
Very Long (>30)	27	0.63	0.70	0.78

Table 8: Accuracy by question length group across all models.

show improved accuracy compared to the long group. This may be because very long answers are predominantly *Ambivalent* (as noted in Section 2.2), making them easier to classify by default. For question length, BERT shows a steady decline in accuracy as questions get longer, while DeBERTa performs best on very long questions (0.78 accuracy), suggesting that its disentangled attention mechanism handles longer and more complex questions more effectively than the other models.

6.3. Concrete Error Examples

To better understand the nature of model failures, we examined specific misclassified examples from the validation set. Below we present representative cases that illustrate the key challenges of this task.

Example 1: Clear Reply predicted as Ambivalent (BERT, DistilBERT).

Question: “Was it a pure coincidence that the payment was sent at the exact time when the American prisoners were released, after being held up for almost four decades?”

Answer (truncated): “Okay. The-it’s been interesting to watch this story surface. Some of you may recall, we announced these payments in January, many months ago. There wasn’t a secret...”

The question asks whether a payment to Iran coincided with a prisoner release. The politician provides a detailed, multi-paragraph response directly addressing the question, explaining the payment’s history and denying any connection to hostages. Despite being a clear and direct reply, the answer’s length (over 300 words) and its use of indirect language (e.g., “it’s been interesting to watch this story surface”) likely confused the model into predicting Ambivalent.

Example 2: Clear Reply predicted as Ambivalent (DistilBERT, DeBERTa).

Question: “Advice to Democrats who might see the strategy as a path to taking back power that the Republicans employed.”

Answer (truncated): “Well, I’ll work in reverse. I think Nancy Pelosi is an outstanding and historic political leader. So much of what we accomplished was accomplished because of her smarts, her tenacity, her legislative skill...”

The response does answer the question by implicitly recommending Pelosi’s con-

tinued leadership. However, the indirect framing and lengthy elaboration make it appear ambivalent.

Example 3: Ambivalent predicted as Clear Reply (BERT, DistilBERT).

Question: “What date would every person in America be able to get a vaccine?”

Answer (truncated): “I think it would be very soon. I think our distribution process is going to go very quickly. I told you, we’ve engaged the military. We have a general who’s in charge of all logistics, ...”

The question asks for a specific date, but the politician never provides one, instead offering general promises (“very soon”, “very quickly”). Despite clearly avoiding the question, the confident tone and detailed discussion of logistics led the models to predict Clear Reply. This illustrates the opposite pattern from Examples 1–2: here, confident language masks the lack of an actual answer.

These examples reveal two recurring patterns. In Examples 1–2, politicians provide long, detailed responses that directly address the question but use indirect or elaborate language, leading the models to misclassify them as Ambivalent. In Example 3, the opposite occurs: a confident tone and detailed but irrelevant content lead the models to predict Clear Reply despite the question being unanswered. In both cases, the models rely on surface-level signals such as answer length, tone, and vocabulary rather than evaluating whether the actual question was addressed.

6.4. Model-Specific Failure Modes

To examine whether the three models fail differently, we compared their misclassification patterns from Table 6. All three models share the same dominant error: confusing *Ambivalent* with *Clear Reply*, which accounts for the largest number of mistakes in every case. However, there are notable differences.

DistilBERT makes the most *Clear Reply* → *Ambivalent* errors (62 vs BERT’s 51 and DeBERTa’s 56), suggesting that its reduced capacity (6 layers vs 12) makes it harder to recognize when a question has been properly answered. In contrast, DeBERTa makes the most *Ambivalent* → *Clear Reply* errors (90 vs BERT’s 88 and DistilBERT’s 79), indicating that it tends to be too confident in classifying responses as clear. On the positive side, DeBERTa achieves the best recall on *Clear Non-Reply* (0.75), meaning it is the most capable of detecting complete evasions.

Overall, the three models fail on largely the same examples, and increasing model capacity shifts the error distribution rather than eliminating errors entirely. The fundamental challenge of distinguishing partial from full responses remains difficult regardless of model size.

6.5. Fundamental Sources of Error and Ideal Model

The core difficulty of this task lies in the nature of political communication itself. Politicians often give responses that sound like answers without actually addressing the question directly, making the boundary between *Clear Reply* and *Ambivalent* fundamentally hard to define. Our analysis identifies three main sources of error:

First, the models rely on surface-level features such as answer length, tone, and vocabulary rather than assessing whether the question was actually addressed. A long, confident response is often predicted as Clear Reply even when the actual question

remains unanswered (Example 3), and a detailed but indirect answer is predicted as Ambivalent even when it does address the question (Examples 1–2).

Second, truncation at 512 tokens removes potentially important context from longer answers. Although we tested 768 tokens for DeBERTa, this decreased performance ($0.67 \rightarrow 0.65$), suggesting that simply increasing the context window is not sufficient and a more targeted approach to capturing relevant information may be needed.

Third, the class imbalance (59% Ambivalent) creates a persistent bias. Despite class weighting, the models still tend to default to Ambivalent when uncertain, as reflected in the high number of misclassifications into this class across all models.

An ideal model for this task would need to go beyond surface-level pattern matching and develop a form of pragmatic understanding: the ability to determine whether the core intent of the question was addressed, regardless of how the answer is framed. This would likely require longer context windows to process full answers without truncation, mechanisms that directly compare what was asked with what was answered, and possibly multi-turn context from the broader interview to resolve ambiguous cases.

7. Discussion

7.1. Results Analysis

Among the three required models, BERT and DeBERTa achieved similar validation Macro F1 scores (0.677 and 0.669 respectively), despite significant differences in vocabulary size (30K vs 128K tokens) and total parameters (109M vs 184M). DistilBERT trailed at 0.626, and this ranking aligns with model capacity: BERT and DeBERTa both have 12 transformer layers, while DistilBERT has only 6. However, on the test set, despite the promising validation Macro F1 scores and small generalization gaps, all models showed a notable drop in Macro F1 (BERT: 0.61, DeBERTa: 0.57, DistilBERT: 0.55) while Weighted F1 remained stable. This gap indicates that the models learned to classify the majority class (*Ambivalent*) well, but their ability to handle minority classes (*Clear Reply* and *Clear Non-Reply*) did not transfer well to unseen data. Training and inference times were consistent with model capacity: DistilBERT was the fastest at 28 seconds per epoch, followed by BERT at 56 seconds and DeBERTa at 87 seconds.

The drop in Macro F1 but not Weighted F1 is directly connected to our dataset and splitting strategy. With only 509 validation samples from a random (non-stratified) split, the minority classes have very few representatives. When we tuned hyperparameters based on validation Macro F1, we may have optimized for patterns specific to those few minority samples rather than learning general patterns. On the test set, which contains different examples, these patterns do not hold.

Hyperparameter tuning produced the largest improvement across all models, confirming that default settings are not suitable for fine-tuning on small, imbalanced datasets. Class weighting helped BERT and DeBERTa but not DistilBERT, likely because DistilBERT's reduced capacity makes it harder to learn minority class patterns even when they receive higher loss weights. Weight decay (0.01) helped BERT and DistilBERT by reducing overfitting, but DeBERTa performed best without it, suggesting that its disentangled attention mechanism already provides enough regularization.

Despite all optimization efforts, the models still overfit. The fundamental reason is the small dataset size (2,881 training samples) combined with the large number of

model parameters (67M-184M). With so many parameters and so few examples, the models inevitably memorize training patterns rather than learning general rules. This is a known limitation of fine-tuning large pretrained models on small datasets, and hyperparameter tuning alone can only partially address it.

We also tested larger model variants (BERT-large, RoBERTa-base, RoBERTa-large, DeBERTa-v3-large) to investigate whether additional capacity improves performance. DeBERTa-v3-large achieved the highest validation Macro F1 among all models (0.69), confirming that larger models can help, but the gains were modest compared to the increase in training time and memory requirements. Notably, RoBERTa-large (355M params) did not outperform RoBERTa-base (125M params), suggesting that simply scaling up model size does not guarantee better results on this task.

Finally, it is worth noting that many of the observed differences between experiments were small (often within 1–2% Macro F1). At this scale, individual runs can be sensitive to random initialization and training order, meaning that a technique like warmup or class weighting may show a small improvement in one run but not in another. This makes it difficult to draw strong conclusions about which individual technique is most effective, and suggests that the overall experimental pipeline (from v0 baseline to v2 hyperparameter tuning) matters more than any single optimization step.

7.2. Comparison with Classical ML Approaches

In our earlier work on this dataset, we applied classical machine learning approaches: TF-IDF and GloVe word embeddings combined with Logistic Regression. Table 9 compares the best results from both projects.

Model	Val Macro F1	Test Macro F1
TF-IDF + LR (best, HW1)	0.63	0.41
GloVe + LR (best, HW1)	0.49	0.43
BERT (best, HW2)	0.68	0.61
DistilBERT (best, HW2)	0.63	0.55
DeBERTa (best, HW2)	0.67	0.57

Table 9: Comparison of classical ML (HW1) and transformer (HW2) approaches.

The improvement from transformers is most visible on the test set, where even the weakest transformer (DistilBERT, 0.55) outperforms the best classical model (0.43) by a significant margin. On the validation set the gains are more modest, with BERT (0.68) only slightly exceeding TF-IDF + LR (0.63). Note that the two projects used different train/val splits, so the validation scores are not directly comparable. The test set comparison is more reliable since both projects evaluated on the same 308 samples.

Despite the clear advantage of transformers, classical models were significantly faster to train (seconds vs minutes) and required no GPU resources, making them a practical choice when computational resources are limited or a quick baseline is needed.

7.3. Submitted Configuration

All experiments reported so far were conducted on Google Colab Pro using an NVIDIA A100-SXM4-40GB GPU, which provided sufficient memory and compute for hyperparameter search, class-weighting ablations, warmup experiments, and input formulation tests without hardware constraints. For the Kaggle submission, the notebooks were re-executed on Kaggle’s NVIDIA T4 $\times 2$ environment. The T4 has only 16 GB of memory per device (compared to the A100’s 40 GB) and offers significantly less compute power than the A100, which introduced a number of hardware-driven constraints that required adaptation of the Colab-validated pipeline.

For BERT-base and DistilBERT-base, no structural changes were required and the Kaggle runs reproduced Colab results within 1–2%, consistent with single-seed run-to-run variance. DeBERTa-v3-base, however, did not fit in T4’s 16 GB memory under the original Colab configuration (batch size 16, fp32 precision, max_length 512), failing with CUDA out-of-memory errors during the v1 training stage. Three modifications were introduced to make DeBERTa-v3-base trainable on T4 while preserving the scientific setup as closely as possible:

1. **Batch size reduction.** For the v0 baseline and v1 class-weighting runs, batch size was reduced from 16 to 8. The final retrain retained batch size 16 (from the HP search’s best configuration) under the memory-saving techniques below. Batch size has only a small effect on final F1 for transformer fine-tuning at this dataset scale.
2. **fp16 mixed precision.** The functions `train_step` and `val_step` were wrapped with `torch.amp.autocast(device_type="cuda", dtype=torch.float16)` and a `GradScaler` was introduced for gradient scaling, following the official PyTorch AMP recipe [7]. Mixed precision reduces activation memory substantially and benefits from the T4’s fp16 tensor cores. The v0 baseline was kept in fp32 for direct comparability with the Colab baseline. All subsequent runs used fp16.
3. **Gradient checkpointing.** After fp16 alone proved insufficient to fit DeBERTa-v3-base on T4 (fp16 OOMed mid-training, likely due to the larger activation memory usage of DeBERTa-v3’s disentangled attention), gradient checkpointing was enabled via `model.gradient_checkpointing_enable()`. Gradient checkpointing saves memory by not storing intermediate activations during the forward pass, recomputing them during the backward pass. The combination of fp16 and gradient checkpointing allowed DeBERTa-v3-base to train successfully on T4, at the cost of slower per-epoch training time.

Additionally, explicit `del model; gc.collect(); torch.cuda.empty_cache()` cleanup was inserted between consecutive training runs (v0, v1, final) to prevent GPU memory fragmentation from causing unexpected OOM errors on later runs. Under these adaptations, DeBERTa-v3-base’s Kaggle final retrain achieved a Macro F1 of 0.640, approximately 3% below the Colab final (0.669). This gap is likely caused by the combined effect of fp16 numerical differences, single-seed variance, gradient checkpointing’s slight impact on gradient flow, and potential hardware-level numerical differences between Kaggle’s T4 and Colab’s A100 (different CUDA kernel implementations and floating-point accumulation schemes across GPU architectures).

Kaggle’s free tier offers a weekly GPU quota of 30 hours, which constrained the number of experimental runs that could be performed directly on the platform. For

this reason, all experiments were first developed and validated on Google Colab Pro, which provides access to high-performance GPUs such as the A100 without Kaggle’s strict weekly quota. The final configurations were re-executed on Kaggle for submission. During the Kaggle re-execution phase, the P100 GPU was also tested but proved incompatible with the required software dependencies, making it unusable for training. The T4 $\times 2$ configuration was used for all final submissions.

Among the required base models, the best Kaggle submission achieved a leaderboard (weighted) F1 of **0.66**, obtained by BERT with its final configuration (Table 2). All three Kaggle notebooks are submitted on Kaggle. The corresponding Colab notebooks, with complete outputs from all experiments, are available in the following **GitHub repository**.

7.4. Future Work

Several directions could potentially improve performance on this task. On the data side, collecting more training samples would help reduce overfitting, which remains the main limitation with only 2,881 training examples. Exploring more advanced approaches to handle class imbalance, such as oversampling minority classes, data augmentation, or synthetic example generation, could help the models learn better representations for underrepresented classes.

On the modeling side, testing alternative validation set sizes (e.g., 10% or 20%) could affect hyperparameter selection stability. Applying custom weight initialization to the classification head, rather than relying on the default random initialization, may lead to faster and more consistent convergence. Testing alternative optimizers beyond AdamW could also reveal different training dynamics.

For DeBERTa specifically, testing intermediate `max_length` values between 512 and 768 (e.g., 640) could help capture more context without introducing the noise observed at 768. Additionally, exploring even larger models such as DeBERTa-v3-large with a full optimization pipeline (class weighting, warmup, input formulation) could push performance further, as the preliminary experiments reported in Appendix ?? show promising results even without extensive tuning.

Finally, approaches beyond fine-tuning could be explored, such as ensemble methods combining predictions from multiple models, or few-shot prompting with large language models to better handle the naturally unclear boundary between *Clear Reply* and *Ambivalent*.

References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.
- [2] Pengcheng He, Jianfeng Gao, and Weizhu Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. *arXiv preprint arXiv:2111.09543*, 2023.

- [3] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *Proceedings of the 9th International Conference on Learning Representations (ICLR)*, 2021.
- [4] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2026. Online manuscript released January 6, 2026.
- [5] Manolis Koubarakis. Course slides: Transformers, BERT, 2026. Artificial Intelligence II Course, National and Kapodistrian University of Athens.
- [6] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, 2019.
- [7] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.
- [8] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*. 2019.
- [9] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- [10] Scikit-learn Developers. Scikit-learn: Machine learning in Python. Accessed: 2026.
- [11] Konstantinos Thomas, Giorgos Filandrianos, Maria Lymperaiou, Chrysoula Zerva, and Giorgos Stamou. “I never said that”: A dataset, taxonomy and base-lines on response clarity classification. *arXiv preprint arXiv:2409.13879*, 2024.
- [12] Konstantinos Thomas, Giorgos Filandrianos, Maria Lymperaiou, Chrysoula Zerva, and Giorgos Stamou. CLARITY: SemEval 2026 task 6, 2026.
- [13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [14] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Barburescu, Morgan Besse, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.

Model	Params	Vocab Size	BS
bert-base	109M	30,522	16
bert-large	335M	30,522	8
roberta-base	125M	50,265	16
roberta-large	355M	50,265	16
deberta-v3-base	184M	128,000	16
deberta-v3-large	435M	128,000	8

Table 10: Architecture details of all tested models.

Model	LR	Epoch	Train Time (s)	Avg. Inf. Time (s)	Macro F1	Weighted F1
bert-base	5e-5	2	238	5.96	0.63	0.64
bert-large	1e-5	2	796	20.64	0.67	0.67
roberta-base	5e-5	3	238	5.97	0.65	0.68
roberta-large	1e-5	4	751	19.52	0.64	0.65
deberta-v3-base	2e-5	4	518	9.92	0.67	0.67
deberta-v3-large	1e-5	3	1365	31.52	0.69	0.70

Table 11: Best results per model variant in the larger model comparison experiment. The bert-base result here (Macro F1 0.63) corresponds to this specific comparison context and differs from the best configuration in Table 3, which achieved 0.677.

A. Additional Experiments

To investigate the effect of model capacity, we tested larger variants alongside the base models. Tables 10 and 11 show the architecture details and best results per model from this comparison.

Table 11 reveals several insights beyond the summary in Section 7. Among the base-scale models, RoBERTa-base achieves a strong Macro F1 of 0.65 with training time identical to bert-base (238s), making it the most efficient upgrade. bert-large and deberta-v3-base reach the same Macro F1 (0.67), but deberta-v3-base trains faster (518s vs 796s) while using a more sophisticated attention mechanism, suggesting that architectural improvements can be more effective than simply increasing model depth. The underperformance of RoBERTa-large (0.64) relative to RoBERTa-base (0.65) is noteworthy and likely reflects insufficient hyperparameter tuning at that scale rather than a fundamental capacity limitation, since larger models generally require lower learning rates and more careful regularization. Finally, deberta-v3-large achieves the best overall result (0.69) but at approximately $5\times$ the training cost of bert-base, raising the question of whether such improvements are worth the additional cost in resource-constrained settings.

GPT-3.5 Summary as Answer Representation

We explored an alternative input representation for BERT where the full interview answer was replaced by the GPT-3.5 generated summary available in the dataset. The motivation was that long answers are frequently truncated at 512 tokens, and a concise summary might preserve the most relevant content while reducing noise. Using the best BERT configuration, this approach achieved a Macro F1 of 0.67, compared to

0.68 with the full answer. This drop suggests that the summarization process, while reducing length, also discards pragmatic signals that are informative for clarity classification. This experiment was conducted only for BERT, as the full answer consistently outperformed the summary representation.

B. Hyperparameter Search Space

Table 12 shows the hyperparameter values explored for each model.

Model	LR	WD	BS	Max Len	Other
BERT-base	5e-5, 3e-5, 2e-5	0.0, 0.01	16	512	warmup, input form.
DistilBERT-base	5e-5, 3e-5, 2e-5	0.0, 0.01	16	512	warmup, input form.
DeBERTa-v3-base	3e-5, 2e-5, 1e-5	0.0, 0.01	8, 16	512, 768	warmup, input form.
BERT-large	5e-5, 3e-5, 1e-5	0.0, 0.01	8, 16	512	–
RoBERTa-base	5e-5, 3e-5, 1e-5	0.0, 0.01	8, 16	512	–
RoBERTa-large	5e-5, 3e-5, 1e-5	0.0, 0.01	8, 16	512	–
DeBERTa-v3-large	3e-5, 2e-5, 1e-5	0.0, 0.01	8, 16	512, 768	–

Table 12: Hyperparameter search space per model. All models used class weighting. Additional models (bottom) were initialized with the best configuration found for their corresponding base model.